# WORKSHEET 1

NAME: KISHORI KAMALI M

REG NO:- 23MIC0055

SUBJECT: ADVAMCE PYTHON PROGRAMMING

QUESTION 1:-

**Problem #1: Design a Python class called Student to track a student's academic performance.**

**Requirements:**

Constructor (_init_)

Accepts two parameters: name (string) and student_id (string or int).

Initializes an empty list scores to store the student's marks.

**Instance Variables:**

name → student's name

student_id → student's unique ID

scores → list of numeric scores (initially empty)

**Methods:**

add_score(score) → Appends a new score (int/float) to the scores list.

calculate_average() → Returns the average of all scores. If no scores exist, return 0.

get_final_grade() → Uses the average to determine result:

"Pass" if average ≥ 60

"Fail" otherwise

**Test Cases**

**1. Object Creation**

```
student1 = Student("Alice", 101)
print(student1.name)        # Expected: Alice
print(student1.student_id)  # Expected: 101
```

print(student1.scores)  # Expected: []

**2. Adding Scores**

student1.add_score(75)

student1.add_score(85)

student1.add_score(55)

print(student1.scores)  # Expected: [75, 85, 55]

**3. Calculate Average**

print(student1.calculate_average())

# Expected: (75 + 85 + 55) / 3 = 71.67

**4. Final Grade (Pass Case)**

print(student1.get_final_grade())

# Expected: Pass (average = 71.67 ≥ 60)

**5. Final Grade (Fail Case)**

student2 = Student("Bob", 102)

student2.add_score(40)

student2.add_score(50)

print(student2.calculate_average())  # Expected: 45.0

print(student2.get_final_grade())  # Expected: Fail

**6. Empty Scores Case**

student3 = Student("Charlie", 103)

print(student3.calculate_average())  # Expected: 0

print(student3.get_final_grade())  # Expected: Fail

CODE:-

```
class Student:#class name

    def __init__(self, name, student_id):# constructor_name
        self.name = name
        self.student_id = student_id
```

```
        self.scores = []
    def add_score(self, score):#method to add score of the students
        self.scores.append(score)
    def calculate_average(self):#method to calcualte the avg of the score
        if len(self.scores) == 0:#if it is a empty list
            return 0
        return sum(self.scores) / len(self.scores)#calculating the avg score and returning
    def get_final_grade(self):#method to check the pass or fail
        average = self.calculate_average()#calling the method to calculate the average
        if average >= 60:
            return "Pass"
        else:
            return "Fail"
student1 = Student("Alice", 101)#objecyion creation
print(student1.name)
print(student1.student_id)
print(student1.scores)

student1.add_score(75)
student1.add_score(85)
student1.add_score(55)#appending the scores
print(student1.scores)

print(student1.calculate_average())

print(student1.get_final_grade())
student2 = Student("Bob", 102)
student2.add_score(40)
student2.add_score(50)
print(student2.calculate_average())
```

print(student2.get_final_grade())

student3 = Student("Charlie", 103)

print(student3.calculate_average())# no mark is added so for thsi instances the avg would be 0

print(student3.get_final_grade())

OUTPUT:-

```
print(student3.get_final_
Alice
101
[]
[75, 85, 55]
71.66666666666667
Pass
45.0
Fail
0
Fail
```

QUESTION 2:-

**Problem #2: Design a Python class called Gene to represent a DNA sequence.**

The class should:

Store a gene's ID and sequence.

Provide methods to:

Print the gene's ID and sequence.

Count the occurrence of a specific base (A, T, G, C).

Calculate the GC content (fraction of G and C bases).

Get and set the sequence, ensuring that RNA bases (U) are not allowed.

**Test Cases**

**1. Object Creation**

gene1 = Gene(101, "ATGCGT")

# Expected output (from constructor prints):

# I'm a new Gene object!

# My constructor got a param: 101

# Assign that param to my id instance variable...

# Similarly, assigning to my sequence instance variable...

**2. Print ID and Sequence**

gene1.print_id()

# Expected: My id is: 101

gene1.print_sequence()

# Expected: My sequence is: ATGCGT

**3. Base Composition**

print(gene1.base_composition("A"))

# Expected: 1

print(gene1.base_composition("G"))

# Expected: 2

print(gene1.base_composition("T"))

# Expected: 1

**4. GC Content**

print(gene1.gc_content())

# Sequence: ATGCGT → G=2, C=1 → GC count=3

# Length=6 → Expected: 3/6 = 0.5

**5. Get Sequence**

print(gene1.get_seq())

# Expected: ATGCGT

**6. Set Sequence (Valid DNA)**

gene1.set_seq("ATATGC")

print(gene1.get_seq())

# Expected: ATATGC

**7. Set Sequence (Invalid RNA)**

gene1.set_seq("AUGC")

# Expected: AssertionError → "Sorry, no RNA allowed."

CODE:-

```
class Gene:#class name


    def __init__(self, gene_id, sequence):#constructor


        print("I'm a new Gene object!")
        print("My constructor got a param:", gene_id)
        print("Assign that param to my id instance variable...")
        print("Similarly, assigning to my sequence instance variable...")
        self.id = gene_id
        self.sequence = sequence
    def print_id(self):#print gene id
        print("My id is:", self.id)


    def print_sequence(self):#print sequence
        print("My sequence is:", self.sequence)


    def base_composition(self, base):#count occurence of a base
        return self.sequence.count(base)


    def gc_content(self):
        gc = self.sequence.count("G") + self.sequence.count("C")
        return gc / len(self.sequence)


    def get_seq(self):
        return self.sequence


    def set_seq(self, new_sequence):
        assert "U" not in new_sequence, "Sorry, no RNA allowed." #it raises the error with the
messagee if the condition is false
        self.sequence = new_sequence
```

```
gene1 = Gene(101, "ATGCGT")

gene1.print_id()#trying to print the id of this instances
gene1.print_sequence()#trying to print the sequence which is stored in this instance
print(gene1.base_composition("A"))
print(gene1.base_composition("G"))
print(gene1.base_composition("T"))
#trying to print the no of occurence of these letters in the sequnece
print(gene1.gc_content())

print(gene1.get_seq())
gene1.set_seq("ATATGC")#updating the sequence
print(gene1.get_seq())#trying to print the updated sequences


try:
    gene1.set_seq("AUGC")#checking whether it accepts the base u and printing the error message


except AssertionError as e:
    print("Error:", e)
```

OUTPUT:-

```
I'm a new Gene object!
My constructor got a param: 101
Assign that param to my id instance variable...
Similarly, assigning to my sequence instance variable...
My id is: 101
My sequence is: ATGCGT
1
2
2
0.5
ATGCGT
ATATGC
```

ASSERTION ERROR

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[2], line 44
     42 gene1.set_seq("ATATGC")#updating the sequence
     43 print(gene1.get_seq())#trying to print the updated sequences
---> 44 gene1.set_seq("AUGC")#checking whether it accepts the base u and printing the error message

Cell In[2], line 28, in Gene.set_seq(self, new_sequence)
     27 def set_seq(self, new_sequence):
---> 28     assert "U" not in new_sequence, "Sorry, no RNA allowed." #it raises the error with the messagee if the condition is false
     29     self.sequence = new_sequence

AssertionError: Sorry, no RNA allowed.
```

ERROR HANDLING:-

```
try:
    gene1.set_seq("AUGC")#checking whether it accepts the base u and printing the error message

except AssertionError as e:
    print("Error:", e)
```

OUTPUT:-

```
I'm a new Gene object!
My constructor got a param: 101
Assign that param to my id instance variable...
Similarly, assigning to my sequence instance variable...
My id is: 101
My sequence is: ATGCGT
1
2
2
0.5
ATGCGT
ATATGC
Error: Sorry, no RNA allowed.
```

# 23MIC0055

# KISHORIKAMALI M

# using a threads in a sequential order

In [ ]:
```python
#import concurrent.futures
import threading #using a threads in a sequential order
import time

start=time.perf_counter()

def do_something():
    seconds=1
    print(f'sleeping {seconds} second(s)')
    time.sleep(1)
    print('done sleeping..')

t1=threading.Thread(target=do_something)

t1.start()#start
t1.join()
t2=threading.Thread(target=do_something)

t2.start()
t2.join()
finish=time.perf_counter()
print(f'Finished in{round(finish-start,2)} seconds(s)')
```

Here because of executing the threads sequentially the overall ime is higher comapartively with parallel processing this is the drawback of executing threads in sequentially

# doing multithreading usinf for loop for parralel processing

In [7]:
```
import threading #doing multithreading usinf for loop for parralel processing
import time

start=time.perf_counter()

def do_something():#function defintion
    seconds=1
    print(f'sleeping {seconds} second(s)')
    time.sleep(1)#making sleep for 1 second
    print('done sleeping..')
threads=[]
for _ in range(10):
    t1=threading.Thread(target=do_something)
    t1.start()
    threads.append(t1)
for i in threads:
    i.join()
finish=time.perf_counter()
print(f'Finished in{round(finish-start,2)} seconds(s)')
```

```
sleeping 1 second(s)
sleeping 1 second(s)
sleeping 1 second(s)
sleeping 1 second(s)
sleeping 1 second(s)
sleeping 1 second(s)
sleeping 1 second(s)
sleeping 1 second(s)
sleeping 1 second(s)
sleeping 1 second(s)
done sleeping..
done sleeping..
done sleeping..
done sleeping..
done sleeping..
done sleeping..
done sleeping..
done sleeping..
done sleeping..
done sleeping..
done sleeping..
Finished in1.0 seconds(s)
```

using thread we acheived parallel processing because of that it took less time when compared with sequntial processing of threads and in sequential threading it took 2 seconds but in parallel it took 1 second because all the two threads started parallel excuted parallely so it took less time

# doing multithreading usinf for loop for parralel processing

In [3]:
```python
import threading #doing multithreading usinf for loop for parralel processing
import time

start=time.perf_counter()

def do_something(seconds):#passing arguments
    print(f'sleeping {seconds} second(s)')
    time.sleep(seconds)#passing the arguementvalue here
    print('done sleeping..')
threads=[]
```

```
for _ in range(10):
    t1=threading.Thread(target=do_something,args=[2])
    t1.start()
    threads.append(t1)
for i in threads:
    i.join()
finish=time.perf_counter()
print(f'Finished in{round(finish-start,2)} seconds(s)')
```

```
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
sleeping 2 second(s)
done sleeping..done sleeping..
done sleeping..

done sleeping..
done sleeping..
done sleeping..
done sleeping..
done sleeping..
done sleeping..
done sleeping..
Finished in2.0 seconds(s)
```

in the last snippet we done parallel processing using for loop and here the same thing but there we used default as 1 second for each thread to get sleep but here i am passing the arguement and the thread gets slepped by the value of that passed arguement last snipped slept for 1 sec and in this snippet i am passing 2 as arguement so it slept for 2 seconds

# doing multithreading using concurrent futures with the help of threadpool executor

In [16]:
```
import concurrent.futures #doing multithreading using concurrent futures with the help of threadpool executor

start=time.perf_counter()

def do_something(seconds):#function defintion

    print(f'sleeping {seconds} second(s)')
    time.sleep(1)#making sleep for 1 second
    print('done sleeping..')
with concurrent.futures.ThreadPoolExecutor() as executor:
    f1=executor.submit(do_something,1.5)
    print(f1.result())
```

```
sleeping 1.5 second(s)
done sleeping..
None
```

in this example we are using concurrent futures for parallel processing of threads because using concurrent futures it took less line of codes

In [13]:
```
f=f1.result()
print(type(f))
```

```
<class 'NoneType'>
```

printing the type of the result

In [15]:
```
s=f1.submit(do_something,1.5)
print(type(s))
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_18175/3020182961.py in <module>
----> 1 s=f1.submit(do_something,1.5)
      2 print(type(s))

AttributeError: 'Future' object has no attribute 'submit'
```

# doing multithreading using concurrent futures with the help of threadpool executor and functio with a single arguement

In [47]:
```
import concurrent.futures #doing multithreading using concurrent futures with the help of threadpool executor and functio with

start=time.perf_counter()

def do_something(seconds):#function defintion

    print(f'sleeping {seconds} second(s)')
    time.sleep(seconds)#making sleep for some seconds which is mentioned in the arguements
    return 'done sleeping..'
with concurrent.futures.ThreadPoolExecutor() as execuetor:

    res=[execuetor.submit(do_something,1.5) for _ in range(2)]#using list comprehension i am creating multiple threads
    for f in concurrent.futures.as_completed(res):
       print(f.result())# this will start the excuetion
finish=time.perf_counter()
print(f'Finished in{round(finish-start,2)} seconds(s)')
```

```
sleeping 1.5 second(s)
sleeping 1.5 second(s)
done sleeping..
done sleeping..
Finished in1.5 seconds(s)
```

i am reducing the code by using list comprehension instead of for loop and also using oncureent futures of threadpoolexecutor,submit,result

# doing multithreading using concurrent futures with the help of threadpool executor and with multiple arguements

In [29]:
```
import concurrent.futures #doing multithreading using concurrent futures with the help of threadpool executor and  with multip

start=time.perf_counter()
```

```
def do_something(seconds):#function defintion

    print(f'sleeping {seconds} second(s)')
    time.sleep(seconds)#making sleep for some seconds which is mentioned in the arguements
    return f'done sleeping..{seconds}..'# prints th statemetn with the arguement of that particular thread to differentiate wh
with concurrent.futures.ThreadPoolExecutor() as execuetor:
    secs=[5,7,3,8,1]
    res=[execuetor.submit(do_something,sec) for sec in secs]#using list comprehension i am creating multiple threads
    for f in concurrent.futures.as_completed(res):
       print(f.result())# this will start the excuetion
finish=time.perf_counter()
print(f'Finished in{round(finish-start,2)} seconds(s)')
```

```
sleeping 5 second(s)
sleeping 7 second(s)
sleeping 3 second(s)
sleeping 8 second(s)
sleeping 1 second(s)
done sleeping..1..
done sleeping..3..
done sleeping..5..
done sleeping..7..
done sleeping..8..
Finished in8.01 seconds(s)
```

here i am passing a list of arguement during function call so each thread gets one by one value in the list for eac thread

# doing multithreading using concurrent futures with the help of threadpool executor and map

In [30]:
```
import concurrent.futures #doing multithreading using concurrent futures with the help of threadpool executor and map

start=time.perf_counter()

def do_something(seconds):#function defintion

    print(f'sleeping {seconds} second(s)')
    time.sleep(seconds)#making sleep for some seconds which is mentioned in the arguements
```

```
    return f'done sleeping..{seconds}..'# prints th statemetn with the arguement of that particular thread to differentiate wh
with concurrent.futures.ThreadPoolExecutor() as execuetor:
    secs=[1,7,3,4,5]
    res=execuetor.map(do_something,secs)#using map function i am creating multiple threads and executing it and printing store
    for f in res:
       print(f)# printing the result that stored in res
finish=time.perf_counter()
print(f'Finished in{round(finish-start,2)} seconds(s)')
```

```
sleeping 1 second(s)
sleeping 7 second(s)
sleeping 3 second(s)
sleeping 4 second(s)
sleeping 5 second(s)
done sleeping..1..
done sleeping..7..
done sleeping..3..
done sleeping..4..
done sleeping..5..
Finished in7.01 seconds(s)
```

i am using map function to map the arguement to each thread during function call beacuse of that it make the code simpler

# trying to download videos without using threads and calculating the time it talen to download

In [46]:
```
#trying to download videos without using threads and calculating the time it talen to download
import time
import requests

video_urls=['https://pixabay.com/videos/nature-landscape-lake-aerial-drone-371424/','https://www.istockphoto.com/video/aerial-
t1=time.perf_counter()
for vid in video_urls:
    vid_bytes=requests.get(vid).content#getting the szes of the files
    vid_name=vid.split('/')[4]#spliting the name from the url
    vid_name=f'{vid_name}.mp4'#making the name for each url
    with open(vid_name,'wb') as vid_file:
        vid_file.write(vid_bytes)#for each url it gets executed and gets downloaded
```

```
        print(f'{vid_name} was downloaded...')
t2=time.perf_counter()
print(f'finishes in{t2-t1} seconds')
```

```
nature-landscape-lake-aerial-drone-371424.mp4 was downloaded...
aerial-calm-surface-of-a-lake-in-the-forest-reflecting-the-beautiful-mount-hood-in-gm864526000-241759649?utm_source=pixabay&utm
_medium=affiliate&utm_campaign=sponsored_video&utm_content=adp_topbanner_media&utm_term=nature+landscape.mp4 was downloaded...
nature-sunrise-mountain-trees-and-aerial-view-of-the-forrest-and-beautiful-scenic-in-gm1430945266-473674323?utm_source=pixabay&
utm_medium=affiliate&utm_campaign=sponsored_video&utm_content=adp_topbanner_media&utm_term=nature+landscape.mp4 was downloade
d...
video.mp4 was downloaded...
netherlands-dwingeloo-autumn-nature-370702.mp4 was downloaded...
solar-eclipse-sun-moon-370344.mp4 was downloaded...
golden-horn-beach-spit-sand-sandy-270941.mp4 was downloaded...
ha-giang-vietnam-drone-flycam-4k-367684.mp4 was downloaded...
beach-sand-region-de-murcia-367098.mp4 was downloaded...
finishes in1.0980627669996466 seconds
```

without using threads parallely it took me 1 second to get download of all files

# using threads downloading the video andd calculating how much time it taken to download all the videos

In [44]:
```
import time #using threads downloading the video andd calculating how much time it taken to download all the videos
import requests

#mentioning ll the urls inside video_urls
video_urls=['https://pixabay.com/videos/nature-landscape-lake-aerial-drone-371424/','https://www.istockphoto.com/video/aerial-
t1=time.perf_counter()
def download_urls(vid):#for each url it gets executed and gets downloaded by this function
    start=time.perf_counter()
    vid_bytes=requests.get(vid).content#getting the szes of the files
    vid_name=vid.split('/')[4]#spliting the name from the url
    vid_name=f'{vid_name}.mp4'#making the name for each url
    with open(vid_name,'wb') as vid_file:

        vid_file.write(vid_bytes)
        finish=time.perf_counter()
```

```
        print(f'file name : {vid_name}  was downloaded...')#after it gets doownloaded it prints the message

        print(f'this took {finish-start}   seconds to get downloaded ')#displaying the time it takes it taken for downloading
        print('\n')
with concurrent.futures.ThreadPoolExecutor() as executor:
    executor.map(download_urls,video_urls)
t2=time.perf_counter()
print(f'finishes in{t2-t1} seconds')
```

```
file name : netherlands-dwingeloo-autumn-nature-370702.mp4  was downloaded...file name : ha-giang-vietnam-drone-flycam-4k-36768
4.mp4  was downloaded...
this took 0.1558488810005656   seconds to get downloaded



this took 0.15880907500104513   seconds to get downloaded


file name : beach-sand-region-de-murcia-367098.mp4  was downloaded...
this took 0.16335579299993697   seconds to get downloaded


file name : nature-landscape-lake-aerial-drone-371424.mp4  was downloaded...
this took 0.19235882900102297   seconds to get downloaded


file name : solar-eclipse-sun-moon-370344.mp4  was downloaded...
this took 0.20780088799801888   seconds to get downloaded


file name : golden-horn-beach-spit-sand-sandy-270941.mp4  was downloaded...
this took 0.20694758399986313   seconds to get downloaded


file name : video.mp4  was downloaded...
this took 0.3131588920005015   seconds to get downloaded


file name : nature-sunrise-mountain-trees-and-aerial-view-of-the-forrest-and-beautiful-scenic-in-gm1430945266-473674323?utm_sou
rce=pixabay&utm_medium=affiliate&utm_campaign=sponsored_video&utm_content=adp_topbanner_media&utm_term=nature+landscape.mp4  wa
s downloaded...
this took 0.36684356900150306   seconds to get downloaded


file name : aerial-calm-surface-of-a-lake-in-the-forest-reflecting-the-beautiful-mount-hood-in-gm864526000-241759649?utm_source
=pixabay&utm_medium=affiliate&utm_campaign=sponsored_video&utm_content=adp_topbanner_media&utm_term=nature+landscape.mp4  was d
ownloaded...
this took 0.44683442799941986   seconds to get downloaded
```

```
finishes in0.44883401399783907 seconds
```

# but here it 0.4 seond to download all the files beaucse of using threads parallely

In [ ]:

# 23MIC0055 KISHORI KAMALI M

# Mutlithread with priority queue and lock

In [1]:
```
import queue
import threading
import time

thread_exit_Flag = 0

class sample_Thread(threading.Thread):

    def __init__(self, threadID, name, q):
        threading.Thread.__init__(self)
        self.threadID = threadID
        self.name = name
        self.q = q

    def run(self):
        print("initializing" + self.name)
        process_data(self.name, self.q)
        print("Exiting" + self.name)


def process_data(threadName, q):
    while not thread_exit_Flag:
        queueLock.acquire()

        if not workQueue.empty():
            data = q.get()
            queueLock.release()
            print("%s processing %s" % (threadName, data))
        else:
            queueLock.release()
            time.sleep(1)
```

```
thread_list = ["Thread-1", "Thread-2", "Thread-3"]
name_list = ["A", "B", "C", "D", "E"]

queueLock = threading.Lock()
workQueue = queue.Queue(10)

threads = []
threadID = 1

for thread_name in thread_list:
    thread = sample_Thread(threadID, thread_name, workQueue)
    thread.start()
    threads.append(thread)
    threadID += 1

queueLock.acquire()

for items in name_list:
    workQueue.put(items)

queueLock.release()

while not workQueue.empty():
    pass

thread_exit_Flag = 1

for t in threads:
    t.join()

print("Exit Main Thread")
```

```
initializingThread-1
initializingThread-2
initializingThread-3
Thread-3 processing A
Thread-3 processing B
Thread-3 processing C
Thread-3 processing D
Thread-3 processing E
ExitingThread-1
ExitingThread-3ExitingThread-2

Exit Main Thread
```

# The special part is the queue lock multiple worker threads tasks are shared among threads safely preventing two threads from accessing the queue at the same time

# Printing the Main thread and Worker thread example 1

In [2]:
```python
import threading
import time

# Function executed by the worker thread
def worker():
    for i in range(5):
        print("Working thread:", threading.current_thread().name)
        time.sleep(1)


# Print the name of the main thread
print("Main thread:", threading.current_thread().name)

# Create a worker thread and give it the name "WorkerThread"
t = threading.Thread(target=worker, name="WorkerThread")
```

```
# Start the worker thread; worker() begins running concurrently
t.start()

# Main thread continues its own work while the worker thread is running
for i in range(5):
    print("Main thread working:", threading.current_thread().name)
    time.sleep(1)

# Wait for the worker thread to finish before the main program exits
t.join()
```

```
Main thread: MainThread
Working thread: WorkerThread
Main thread working: MainThread
Working thread: WorkerThread
Main thread working: MainThread
Working thread:Main thread working: MainThread
 WorkerThread
Main thread working: MainThread
Working thread: WorkerThread
Working thread:Main thread working: MainThread
 WorkerThread
```

## The special part is threading.current_thread().name which shows exactly which thread is executing each task MainThread or WorkerThread

## It also uses t.join() to make the main thread wait until the worker thread finishes

## 2nd example

In [3]:
```python
import threading
import time

# Function executed by the background worker thread
def worker_task():
    """Function executed by the background worker thread."""
    thread_name = threading.current_thread().name

    # Worker thread performs 3 steps
    for i in range(1, 4):
        print(f"[{thread_name}] Working on step {i}...")
        time.sleep(1)


# Function executed by the main thread
def main():
    """Function executed by the main thread."""
    main_name = threading.current_thread().name

    print(f"[{main_name}] Starting the application.")

    # Create a worker thread and assign it the name "WorkerThread"
    worker_thread = threading.Thread(
        target=worker_task,
        name="WorkerThread"
    )

    # Start the worker thread
    worker_thread.start()

    # Main thread continues its own work while the worker runs
    for i in range(1, 4):
        print(f"[{main_name}] Managing resources step {i}...")
        time.sleep(0.8)

    # Wait for the worker thread to finish completely
    worker_thread.join()

    print(f"[{main_name}] All threads completed successfully.")
```

```
# Start the program by calling the main function
if __name__ == "__main__":
    main()
```

```
[MainThread] Starting the application.
[WorkerThread] Working on step 1...
[MainThread] Managing resources step 1...
[MainThread] Managing resources step 2...
[WorkerThread] Working on step 2...
[MainThread] Managing resources step 3...
[WorkerThread] Working on step 3...
[MainThread] All threads completed successfully.
```

# the use of this code is to demonstrate mutlithreading in python specially how a main thread and a worker thread prform different tasks concurrently

In [ ]:


1. The Blueprint and Tools
Queue
Threading
Time
0
threadexitFlag = 0
2. The Factory Workers Awaken
Thread-1
Thread-2
Thread-3
(init() method)
3. Booting Up and the Lock Key
queueLock
4. Loading the Inventory
A
B
C
C
D
E
5. Thread-1 Takes the Key
queueLock
6. Processing Data Crate A
Thread-1 processing A
B
A
Concurrent Competition
A
B
B
C
3
The Shutdown Signal
0
1
threadexitFlag = 1
Final Deactivation
Exit Main Thread
Final Deactivation stops upeoon the loops upon sees thay
back their docking bays. Nee by one, their eye lights click